

INSTRUCTION FORMAT

Presented by

**AMAN DEEP SINGH
ASSISTANT PROFESSOR
GLA UNIVERSITY**

INTRODUCTION

- The instruction formats are a sequence of bits (0 and 1). These bits, when grouped, are known as fields. Each field of the machine provides specific information to the CPU related to the operation and location of the data.
- The instruction format also defines the layout of the bits for an instruction. It can be of variable lengths with multiple numbers of addresses. These address fields in the instruction format vary as per the organization of the registers in the CPU. The formats supported by the CPU depend upon the Instructions Set Architecture implemented by the processor.

TYPES OF INSTRUCTION FORMAT

There are mainly four types of Instruction format depending on the multiple address fields which are as follows :

1. Zero-address instructions
2. One-address instructions
3. Two-address instructions
4. Three-address instructions

Zero-address instructions

This instruction does not have an operand field, and the location of operands is implicitly represented. The stack-organized computer system supports these instructions. To evaluate the arithmetic expression, it is required to convert it into reverse polish notation

Example: Consider the below operations, which shows how $X = (A + B) * (C + D)$ expression will be written for a stack-organized computer.

TOS: Top of the Stack

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C + D)$
MUL		$TOS \leftarrow (C + D) * (A + B)$
POP	X	$M[X] \leftarrow TOS$

One-address instructions

This instruction uses an implied accumulator for data manipulation operations. An accumulator is a register used by the CPU to perform logical operations. In one address instruction, the accumulator is implied, and hence, it does not require an explicit reference. For multiplication and division, there is a need for a second register.

Example: The program to evaluate $X = (A + B) * (C + D)$ is as follows:

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow A[C] + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

Two-address instructions

This instruction is most commonly used in commercial computers. This address instruction format has three operand fields. The two address fields can either be memory addresses or registers.

MODE	OPCODE	OPERAND 1	OPERAND 2
------	--------	-----------	-----------

Example: The program to evaluate $X = (A + B) * (C + D)$ is as follows:

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

The MOV instruction transfers the operands to the memory from the processor registers. R1, R2 registers.

Three-address instructions

The format of a three address instruction requires three operand fields. These three fields can be either memory addresses or registers

MODE	OPCODE	OPERAND 1	OPERAND 2	OPERAND 3
------	--------	-----------	-----------	-----------

Example: The program in assembly language $X = (A + B) * (C + D)$ Consider the instructions given below that explain each instruction's register transfer operation.

ADD	R1, A, B	$R1 \leftarrow M[A] + M[B]$
ADD	R2, C, D	$R2 \leftarrow M[C] + M[D]$
MUL	X, R1, R2	$M[X] \leftarrow R1 * R2$

Two processor registers, R1 and R2.

The symbol $M[A]$ denotes the operand at memory address symbolized by A. The operand1 and operand2 contain the data or address that the CPU will operate. Operand 3 contains the result's address.

Advantages and disadvantages of Zero-address instructions

- **Advantages:**
 - They are simple and can be executed quickly since they do not require any operand fetching or addressing. They also take up less memory space.
- **Disadvantages:**
 - They can be limited in their functionality and do not allow for much flexibility in terms of addressing modes or operand types.

Advantages and disadvantages of One-address instructions

Advantages:

They allow for a wide range of addressing modes, making them more flexible than zero-address instructions. They also require less memory space than two or three-address instructions.

Disadvantages:

They can be slower to execute since they require operand fetching and addressing.

Advantages and disadvantages of Two-address instructions

Advantages:

They allow for more complex operations and can be more efficient than one-address instructions since they allow for two operands to be processed in a single instruction. They also allow for a wide range of addressing modes.

Disadvantages:

They require more memory space than one-address instructions and can be slower to execute since they require operand fetching and addressing.

Advantages and disadvantages of Three-address instructions

Advantages:

They allow for even more complex operations and can be more efficient than two-address instructions since they allow for three operands to be processed in a single instruction. They also allow for a wide range of addressing modes.

Disadvantages:

They require even more memory space than two-address instructions and can be slower to execute since they require operand fetching and addressing.

THANK YOU